SentimentScope: Sentiment Analysis using Transformers!

Introduction

In this notebook, you will train a transformer model from scratch to perform sentiment analysis on the IMDB dataset. You are a Machine Learning Engineer at Cinescope, a growing entertainment company working to enhance its recommendation system. Your task is to fine-tune a transformer-based model for sentiment analysis using the IMDB dataset. By classifying reviews as positive or negative, you will help the company better understand user sentiment and deliver more personalized experiences.

By completing this project, you will demonstrate your competency in the following learning objectives:

- Load, explore, and prepare a text dataset for training a transformer model using PyTorch.
- Customize the architecture of the transformer model for a classification task.
- Train and test a transformer model on the IMDB dataset.

Now that you have an overview of what you will achieve in this project, let's move on to the project outline.

Project Outline

This notebook is organized into the following sections:

1. Introduction: Overview of the project, learning objectives, and understanding sentiment analysis.
2. Load, Explore, and Prepare the Dataset: Load the IMDB dataset, explore it with visualizations, and split it into training and validation sets.
3. Implement a DataLoader in PyTorch: Create the `IMDBDataset` class and use it with the PyTorch `DataLoader`, including tokenization.
4. Customize the Transformer Architecture: Modify the transformer model for binary classification.
5. Implement Accuracy Calculation Method: Create a function to compute accuracy for monitoring performance.
6. Train the Model: Complete and execute the training loop for binary classification.
7. Test the Model: Evaluate the model on the test dataset and ensure it achieves over 75% accuracy.
8. Conclusion: Summarize the project results and key takeaways.

Click on the section titles above to navigate directly to the corresponding part of the notebook!

Now that we've outlined the structure and objectives of this project, let's delve into the core concept: sentiment analysis.

Understanding Sentiment Analysis

Sentiment analysis is a natural language processing (NLP) technique used to determine the sentiment expressed in a piece of text. This can range from identifying the polarity (positive, negative, or neutral) of a review to analyzing emotions and opinions.

In this project, sentiment analysis is explicitly framed as a **binary classification task**, where the goal is to determine whether a given movie review is *positive* or *negative*. This task is central to many real-world applications, including customer feedback analysis, social media monitoring, and recommendation systems. By developing a transformer-based model, you will classify IMDB movie reviews as positive or negative to tackle the challenge faced by your entertainment company CineScope by enhancing its recommendation system, enabling more accurate and personalized suggestions.

Reviews labeled as positive will be marked as 1 in the dataset, while negative reviews will be labeled as 0.

For example, consider the following movie review:

> "The movie was a rollercoaster of emotions, and I loved every moment of it!"

This review is clearly positive as it expresses enjoyment and satisfaction with the movie, hence it will be labelled as *positive* or 1 in the dataset. In contrast:

> "The plot was predictable, and the acting was subpar. A waste of time."

This review conveys a negative sentiment, criticizing both the plot and acting, hence it will be labelled as *negative* or 0 in the dataset.

While transformers are often used for generation tasks, they can also be adapted for classification tasks with some modifications to their architecture. You might already be familiar with the tweaks that we will implement in this project.

Data Description

The dataset used in this project is the IMDB dataset, provided in the `aclIMDB_v1.tar.gz` file. Upon extracting the file, you will find the following folder structure:

```
aclIMDB/
├── train/
│   ├── pos/    # Positive reviews for training
│   ├── neg/    # Negative reviews for training
│   ├── unsup/  # Unsupervised data (not used in this project)
├── test/
│   ├── pos/    # Positive reviews for testing
│   ├── neg/    # Negative reviews for testing
```

- **train/**: Contains labeled data for training the model. Reviews in the `pos/` folder should be labeled as positive (1), while reviews in the `neg/` folder should be labeled as negative (0).
- **test/**: Contains labeled data for evaluating the model. Similar to the training data, `pos/` and `neg/` contain positive and negative reviews, respectively.
- **unsup/**: Contains unlabeled reviews that are not used in this project.

Understanding the folder structure is crucial as it guides how we load and preprocess the data for the sentiment classification task.

Load, Explore, and Prepare the Dataset

1. Load the Dataset

The dataset is already available in the environment as `aclIMDB_v1.tar.gz`. We will load it into Pandas DataFrames for easy exploration and preparation.

In [14]:
```
import os
import pandas as pd
```

In [17]:
```
# Unpack the dataset - uncomment the line below to run
!tar -xzf aclImdb_v1.tar.gz
```

You have successfully extracted the folder. Go back to your workspace and explore the folder structure and find the relative paths for each of the following:

- Training positive reviews
- Training negative reviews
- Testing positive reviews
- Testing negative reviews

Assign the paths of these folders relative to the starter file in the variables below.

In [15]:
```
# Define paths to dataset
train_pos_path = "aclImdb/train/pos"
train_neg_path = "aclImdb/train/neg"
```

```
test_pos_path = "aclImdb/test/pos"
test_neg_path = "aclImdb/test/neg"
```

Now, you will implement the `load_dataset()` function, which reads all text files in a specified folder and returns their content as a list of strings. This function is essential for loading and preprocessing the dataset in subsequent steps.

To implement this function:

1. **Use the `os` module**: Leverage Python's `os` module to list all files in the folder.
2. **Handle file paths**: Use `os.path.join()` to construct full paths for files, ensuring compatibility across operating systems.
3. **Read file content**: Open each file in read mode (`'r'`) using UTF-8 encoding to handle text properly.
4. **Aggregate results**: Append the content of each file to a list and return it.

Key Points to Consider:

- Ensure that the function only processes text files (you may use file extensions for filtering if needed).
- Refer to the os.listdir documentation for listing files in a directory.

In [19]:
```
def load_dataset(folder):
    """
    Reads all text files in the specified folder and returns their content as a lis

    Args:
        folder (str): Path to the folder containing text files.

    Returns:
        list: A list of strings, where each string is the content of a text file.
    """
    reviews = []

    for filename in sorted(os.listdir(folder)):
        filepath = os.path.join(folder, filename)

        if filename.endswith(".txt") and os.path.isfile(filepath):
            with open(filepath, "r", encoding="utf-8") as file:
                reviews.append(file.read())

    return reviews
```

Use the function now to load the training and testing data:

In [20]:
```
# Load training and testing data
train_pos = load_dataset(train_pos_path)
train_neg = load_dataset(train_neg_path)
test_pos = load_dataset(test_pos_path)
test_neg = load_dataset(test_neg_path)
```

We can convert the data into pandas dataframes to make handling the datasets easier.

In [21]:
```
# Create DataFrames
train_df = pd.DataFrame({
    'review': train_pos + train_neg,
    'label': [1] * len(train_pos) + [0] * len(train_neg)
})

test_df = pd.DataFrame({
    'review': test_pos + test_neg,
    'label': [1] * len(test_pos) + [0] * len(test_neg)
})

print(train_df.head())
```

```
                                              review  label
0  Bromwell High is a cartoon comedy. It ran at t...      1
1  Homelessness (or Houselessness as George Carli...      1
2  Brilliant over-acting by Lesley Ann Warren. Be...      1
3  This is easily the most underrated film inn th...      1
4  This is not the typical Mel Brooks film. It wa...      1
```

You can ensure that your datasets have loaded correctly by running the following code cell. No output means success!

In [22]:
```
# Assert that both datasets have the expected number of rows
assert train_df.shape[0] == 25000, "Training dataset does not have 25000 rows."
assert test_df.shape[0] == 25000, "Testing dataset does not have 25000 rows."

# Assert that both datasets have exactly two columns
assert train_df.shape[1] == 2, "Training dataset does not have exactly 2 columns."
assert test_df.shape[1] == 2, "Testing dataset does not have exactly 2 columns."
```

2. Explore the Dataset

Exploration helps us understand the dataset's structure and distribution.

Here are some suggestions for exploration and visualizations:

- **Dataset Overview**: Use `DataFrame.info()` and `DataFrame.describe()` to understand the dataset structure and basic statistics.
- **Label Distribution**: Create bar charts to visualize the number of positive and negative reviews.
- **Review Length Analysis**: Compute and plot the distribution of review lengths (e.g., number of characters or words).
- **Sample Reviews**: Print a few positive and negative reviews to understand the text content.

Write code to explore the dataset in this section.

In [27]:
```
# 1. Dataset Overview
print("Training Dataset Information:")
train_df.info()

print("\nTraining Dataset Statistics:")
print(train_df.describe(include="all"))
```

```
Training Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   review         25000 non-null  object
 1   label          25000 non-null  int64 
 2   review_length  25000 non-null  int64 
dtypes: int64(2), object(1)
memory usage: 586.1+ KB

Training Dataset Statistics:
                                                   review        label  \
count                                               25000  25000.00000   
unique                                              24904          NaN   
top     You do realize that you've been watching the E...          NaN   
freq                                                    3          NaN   
mean                                                  NaN      0.50000   
std                                                   NaN      0.50001   
min                                                   NaN      0.00000   
25%                                                   NaN      0.00000   
50%                                                   NaN      0.50000   
75%                                                   NaN      1.00000   
max                                                   NaN      1.00000   

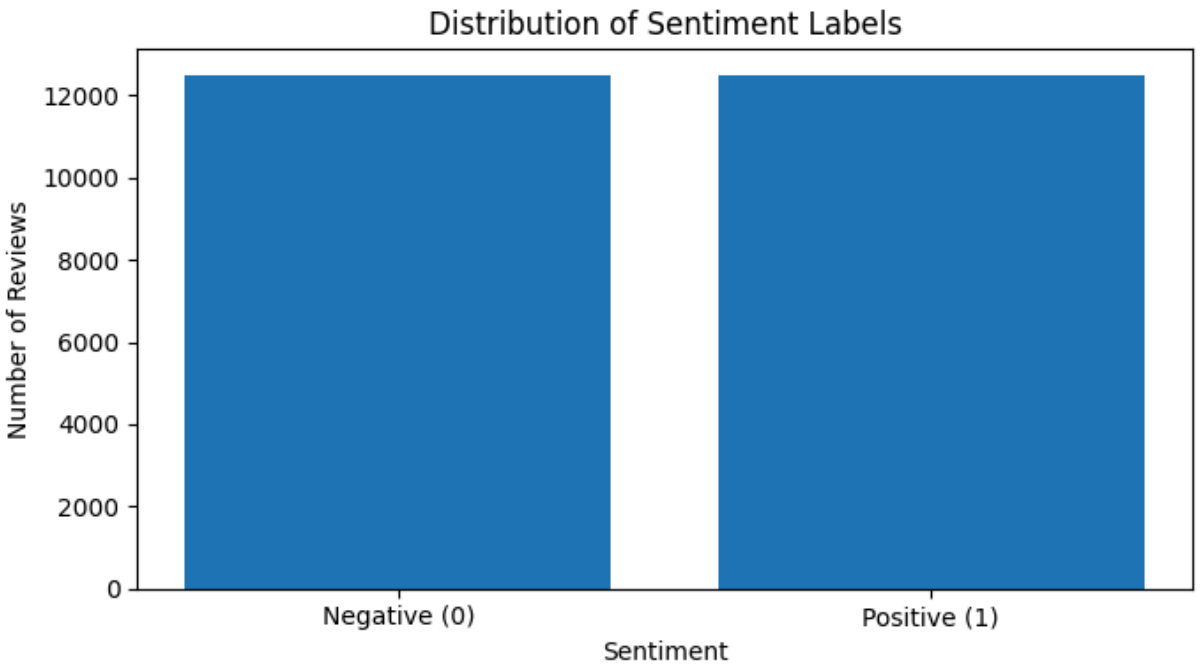
        review_length  
count    25000.000000  
unique            NaN  
top               NaN  
freq              NaN  
mean       233.787200  
std        173.733032  
min         10.000000  
25%        127.000000  
50%        174.000000  
75%        284.000000  
max       2470.000000  
```

In [28]:
```
# 2. Label Distribution
import matplotlib.pyplot as plt

label_counts = train_df["label"].value_counts().sort_index()

plt.figure(figsize=(7, 4))
plt.bar(
    ["Negative (0)", "Positive (1)"],
    [label_counts.get(0, 0), label_counts.get(1, 0)]
)
```

```
plt.title("Distribution of Sentiment Labels")
plt.xlabel("Sentiment")
plt.ylabel("Number of Reviews")
plt.tight_layout()
plt.show()
```

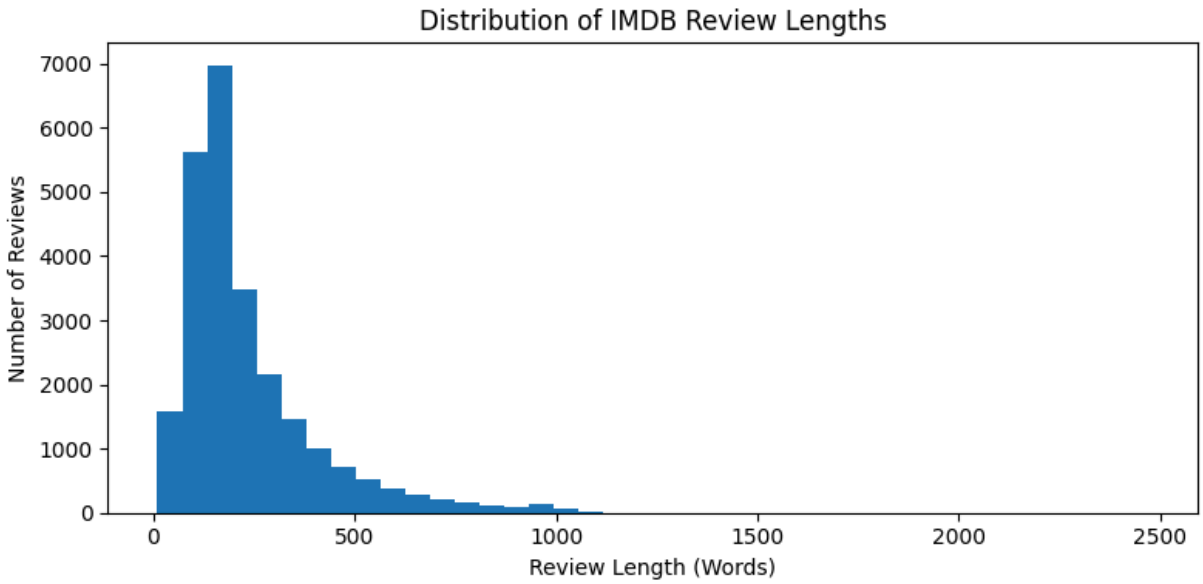


In [29]:
```
# 3. Review Length Analysis
train_df["review_length"] = train_df["review"].apply(
    lambda text: len(text.split())
)

print("\nReview Length Statistics:")
print(train_df["review_length"].describe())

plt.figure(figsize=(8, 4))
plt.hist(train_df["review_length"], bins=40)
plt.title("Distribution of IMDB Review Lengths")
plt.xlabel("Review Length (Words)")
plt.ylabel("Number of Reviews")
plt.tight_layout()
plt.show()
```

```
Review Length Statistics:
count    25000.000000
mean       233.787200
std        173.733032
min         10.000000
25%        127.000000
50%        174.000000
75%        284.000000
max       2470.000000
Name: review_length, dtype: float64
```



In [30]:
```
# 4. Sample Reviews
print("\nPositive Review Sample:")
print(train_df.loc[train_df["label"] == 1, "review"].iloc[0][:500])

print("\nNegative Review Sample:")
print(train_df.loc[train_df["label"] == 0, "review"].iloc[0][:500])
```

```
Positive Review Sample:
Bromwell High is a cartoon comedy. It ran at the same time as some other programs ab
out school life, such as "Teachers". My 35 years in the teaching profession lead me
to believe that Bromwell High's satire is much closer to reality than is "Teachers".
The scramble to survive financially, the insightful students who can see right throu
gh their pathetic teachers' pomp, the pettiness of the whole situation, all remind m
e of the schools I knew and their students. When I saw the episode in which a s

Negative Review Sample:
Story of a man who has unnatural feelings for a pig. Starts out with a opening scene
that is a terrific example of absurd comedy. A formal orchestra audience is turned i
nto an insane, violent mob by the crazy chantings of it's singers. Unfortunately it
stays absurd the WHOLE time with no general narrative eventually making it just too
off putting. Even those from the era should be turned off. The cryptic dialogue woul
d make Shakespeare seem easy to a third grader. On a technical level it's bette
```

3. Prepare the Dataset

We will split the training data further into training and validation subsets. The way we constructed the dataset, reviews with positive and negative labels are segregated. To ensure that the validation dataset works well, we first need to shuffle the dataset.

In [31]:
```
# Split train data into training and validation sets manually
train_size = int(0.9 * len(train_df))

# Shuffle the dataset
shuffled_df = train_df.sample(
    frac=1, random_state=42
).reset_index(drop=True)
```

```
train_data = shuffled_df.iloc[:train_size]
val_data = shuffled_df.iloc[train_size:]
```

4. Testing the Tokenizer

Subword Tokenization

In earlier tasks, you might have encountered character-level tokenization, where each character in the text is treated as a token. While this is straightforward, it is less efficient and may result in larger input sizes, impacting the performance of transformer models.

To address this, we will use Hugging Face's `AutoTokenizer`, a robust and efficient class designed for tokenizing text based on pretrained models. Specifically, we will utilize the `bert-base-uncased` tokenizer, which applies **subword tokenization**. Subword tokenization involves two steps:

1. **Subword Splitting**: Words are split into smaller components (subwords) based on a predefined vocabulary. For example:
    - Input: "unhappiness"
    - Subword Splits: `['un', 'happiness']`
2. **Token Conversion**: Each subword is then converted into a numerical token ID. For example:
    - Subword Splits: `['un', 'happiness']`
    - Token IDs: `[1011, 24123]` (values are illustrative and depend on the tokenizer vocabulary).

About the `bert-base-uncased` Tokenizer

The `bert-base-uncased` tokenizer, developed by Google researchers, is part of the BERT model family. This tokenizer is associated with the `bert-base-uncased` model, which has been widely used for tasks such as sentiment analysis, question answering, and text classification. The tokenizer ensures all text is converted to lowercase and accents are removed, reducing vocabulary size and improving generalization. Example:

- Input: "I Love Transformers."
- Subword Splits: `['i', 'love', 'trans', '##formers', '.']`

You can learn more about the tokenizer and model on Hugging Face's bert-base-uncased page.

Using the `AutoTokenizer` Class

The `AutoTokenizer` class in the Hugging Face Transformers library provides a seamless way to load tokenizers for various pretrained models. It automatically selects the correct

tokenizer configuration based on the model name.

In [34]:
```
from transformers import AutoTokenizer

# Initialize tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

This initializes a tokenizer tailored for the `bert-base-uncased` model. You can refer to the Hugging Face AutoTokenizer documentation for more details. You can find the definition of the `from_pretrained()` method here.

Instead of tokenizing the entire dataset, we will test the tokenizer on a few sample reviews directly from the dataset:

In [35]:
```
from transformers import AutoTokenizer

# Initialize tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Take sample inputs from the dataset
sample_texts = train_data['review'].sample(3, random_state=42).tolist()

# Tokenize sample inputs
tokenized_samples = tokenizer(
    sample_texts,
    truncation=True,
    padding="max_length",
    max_length=128,
    return_tensors="pt"
)

# Display tokenized output
print(tokenized_samples)

# Check output shapes
print("\nInput IDs shape:", tokenized_samples["input_ids"].shape)
print("Attention mask shape:", tokenized_samples["attention_mask"].shape)
```

```
{'input_ids': tensor([[  101,  2348,  1996,  3185,  2003,  4415,  6052,  1010,  950
1,  2064,
          2145,  4089,  6709,  2007,  1996, 24525,  1997,  5292, 21112,  2015,
         18396,  1999,  2023, 27768,  1998,  2200,  6057,  2104, 16168,  6925,
          1012, 18396,  9590,  2114,  4895, 18824,  2135, 10238,  1999,  2093,
          2367,  5535,  1024,  1996,  2962,  2287,  1010,  1996,  3142,  2287,
          1010,  1998,  1996,  5549,  2078,  2287,  1010,  2652,  2471,  1996,
          2168,  2839,  2007,  2074,  1037,  2689,  1997, 17363,  2000,  2393,
          2149,  6709,  1996,  2367,  1000,  5535,  1000,  1012,  1999,  2023,
          3185,  2057,  2156,  2028,  1997,  1996,  5700, 21699, 20818,  1997,
          1996,  1000,  5430,  2386,  1000, 12991, 13874,  1010,  2040,  5222,
          2010,  2293,  2025,  2011,  7472,  2021,  2011, 26128,  2486,  1010,
          2004,  2092,  2004,  1037,  6057,  9792,  2006,  3142,  5580,  2401,
         29469,  2389,  4337,  1010,  2048, 21699, 12741,   102],
        [  101,  1996,  4071,  1997,  2383,  2115,  2219,  2712,  2183,  2373,
          4049,  1010,  1996,  8277,  1997,  2183,  2006, 11564,  7357,  1037,
         17638,  1010,  2019,  1005,  1055,  2158,  1997,  2019, 29506,  1998,
          8403,  1006,  1998,  2061,  2092, 19038,   999,  1007, 16838,  1999,
          2986, 20345,  2015,  2020,  2035, 15298,  4855,  2685,  2005,  1000,
          2712,  5690,  1000,  1006,  3845,  1011,  6079,  1007,  1012,  1026,
          7987,  1013,  1028,  1026,  7987,  1013,  1028,  2074,  2054,  2001,
          1996,  3114,  2005,  5155,  1037,  4066,  1997,  2712,  2183,  1000,
          3282,  2005, 10887,  1000,  1008,  2186,  1012,  2292,  1005,  1055,
          2298,  4876,  2085,  1012,  2045,  2442,  2022,  1037,  2070, 15774,
          2105,  1012,  1026,  7987,  1013,  1028,  1026,  7987,  1013,  1028,
          2065,  2057,  2020,  2000,  2298,  2067,  2074,  1037,  2210,  1010,
          2057,  2156,  1996,  1054,  3683,  2557,  4620,   102],
        [  101,  2045,  2323,  2022,  2062,  5691,  2055,  2256,  3128,  4841,
          1012,  1045,  2926,  2228,  2008,  2478,  5025,  2613,  3128,  4841,
          1010,  2052,  2022,  1996,  1996,  2157,  2518,  1012,  1045,  2113,
          2008,  2023, 13255, 20252,  5420,  1010,  2040,  2001,  2209,  2011,
          9267, 10243,  7229,  2002,  2106,  2019,  6581,  3105,  1999, 17274,
          2008,  2839,  1010,  2144,  2002,  2001,  2019, 25244,  1012,  2021,
          2026, 10293,  2000,  5365,  1010,  2003,  2000,  2404,  2062,  2137,
          6505,  2046,  1996,  4395,  1010,  1998,  2196,  2224,  3087,  2842,
          1012,  1996, 16615,  3842,  2038,  2042,  2404,  2006,  1996,  2067,
          6402,  2121,  2521,  2205,  2146,  1012,  2037,  5635,  2003,  1037,
         29591,  2000,  2256,  2406,  1012,  2009,  2003,  2026,  3813,  6772,
          2008,  2256,  2406,  2323,  2709,  1996,  2304,  4564,  2000,  1996,
         16615,  1012,  2057,  3198,  3956,  2000,  2709,   102]]), 'token_type_id
s': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         
```

```
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1]])}

Input IDs shape: torch.Size([3, 128])
Attention mask shape: torch.Size([3, 128])
```

In [36]:
```
print(tokenized_samples)
```

```
{'input_ids': tensor([[  101,  2348,  1996,  3185,  2003,  4415,  6052,  1010,  950
1,  2064,
          2145,  4089,  6709,  2007,  1996, 24525,  1997,  5292, 21112,  2015,
         18396,  1999,  2023, 27768,  1998,  2200,  6057,  2104, 16168,  6925,
          1012, 18396,  9590,  2114,  4895, 18824,  2135, 10238,  1999,  2093,
          2367,  5535,  1024,  1996,  2962,  2287,  1010,  1996,  3142,  2287,
          1010,  1998,  1996,  5549,  2078,  2287,  1010,  2652,  2471,  1996,
          2168,  2839,  2007,  2074,  1037,  2689,  1997, 17363,  2000,  2393,
          2149,  6709,  1996,  2367,  1000,  5535,  1000,  1012,  1999,  2023,
          3185,  2057,  2156,  2028,  1997,  1996,  5700, 21699, 20818,  1997,
          1996,  1000,  5430,  2386,  1000, 12991, 13874,  1010,  2040,  5222,
          2010,  2293,  2025,  2011,  7472,  2021,  2011, 26128,  2486,  1010,
          2004,  2092,  2004,  1037,  6057,  9792,  2006,  3142,  5580,  2401,
         29469,  2389,  4337,  1010,  2048, 21699, 12741,   102],
        [  101,  1996,  4071,  1997,  2383,  2115,  2219,  2712,  2183,  2373,
          4049,  1010,  1996,  8277,  1997,  2183,  2006, 11564,  7357,  1037,
         17638,  1010,  2019,  1005,  1055,  2158,  1997,  2019, 29506,  1998,
          8403,  1006,  1998,  2061,  2092, 19038,   999,  1007, 16838,  1999,
          2986, 20345,  2015,  2020,  2035, 15298,  4855,  2685,  2005,  1000,
          2712,  5690,  1000,  1006,  3845,  1011,  6079,  1007,  1012,  1026,
          7987,  1013,  1028,  1026,  7987,  1013,  1028,  2074,  2054,  2001,
          1996,  3114,  2005,  5155,  1037,  4066,  1997,  2712,  2183,  1000,
          3282,  2005, 10887,  1000,  1008,  2186,  1012,  2292,  1005,  1055,
          2298,  4876,  2085,  1012,  2045,  2442,  2022,  1037,  2070, 15774,
          2105,  1012,  1026,  7987,  1013,  1028,  1026,  7987,  1013,  1028,
          2065,  2057,  2020,  2000,  2298,  2067,  2074,  1037,  2210,  1010,
          2057,  2156,  1996,  1054,  3683,  2557,  4620,   102],
        [  101,  2045,  2323,  2022,  2062,  5691,  2055,  2256,  3128,  4841,
          1012,  1045,  2926,  2228,  2008,  2478,  5025,  2613,  3128,  4841,
          1010,  2052,  2022,  1996,  1996,  2157,  2518,  1012,  1045,  2113,
          2008,  2023, 13255, 20252,  5420,  1010,  2040,  2001,  2209,  2011,
          9267, 10243,  7229,  2002,  2106,  2019,  6581,  3105,  1999, 17274,
          2008,  2839,  1010,  2144,  2002,  2001,  2019, 25244,  1012,  2021,
          2026, 10293,  2000,  5365,  1010,  2003,  2000,  2404,  2062,  2137,
          6505,  2046,  1996,  4395,  1010,  1998,  2196,  2224,  3087,  2842,
          1012,  1996, 16615,  3842,  2038,  2042,  2404,  2006,  1996,  2067,
          6402,  2121,  2521,  2205,  2146,  1012,  2037,  5635,  2003,  1037,
         29591,  2000,  2256,  2406,  1012,  2009,  2003,  2026,  3813,  6772,
          2008,  2256,  2406,  2323,  2709,  1996,  2304,  4564,  2000,  1996,
         16615,  1012,  2057,  3198,  3956,  2000,  2709,   102]]), 'token_type_id
s': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         
```

```
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1]])}
```

Explanation of parameters:

- `truncation=True` : Truncates text longer than the specified max_length.
- `padding=True` : Pads shorter sequences to match max_length.
- `max_length=128` : Specifies the maximum length of the sequences.
- `return_tensors="pt"` : Returns PyTorch tensors as the output format.

For more details about truncation and padding, refer to the Hugging Face Padding and Truncation Documentation. This step ensures that the tokenizer works as expected and provides insight into its behavior. Next, we will use the tokenizer within the class definition to process the dataset.

Implement a DataLoader in PyTorch

In this section, you will implement a custom dataset class and use it to create a DataLoader in PyTorch for feeding data into the model during training. PyTorch simplifies this process by providing the `Dataset` and `DataLoader` classes, which handle batching, shuffling, and preprocessing, allowing you to focus on the model architecture and training.

To start, we will create a custom dataset class for the IMDB dataset, which will process and return tokenized inputs along with their corresponding labels. This class will use a tokenizer to preprocess the raw text data.

1. Define a Custom Dataset Class

The custom dataset class will inherit from `torch.utils.data.Dataset` and include the following features:

1. **Initialization (`__init__`)**:
    - Accepts raw text and label data, along with a tokenizer and a maximum sequence length.
    - The tokenizer is used to preprocess the text data into tokenized inputs.
    - The maximum sequence length ensures that all tokenized inputs are of uniform length.
2. **Length (`__len__`)**:
    - Returns the total number of data samples in the dataset.
3. **Item Retrieval (`__getitem__`)**:
    - Retrieves a single data point by index.
    - Preprocesses the text using the tokenizer to create tokenized input IDs.
    - Returns the tokenized input IDs and the corresponding label for the given index.

You can refer to this tutorial on the Pytorch website for more details.

In [37]:
```
import torch
from torch.utils.data import Dataset

MAX_LENGTH = 128

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

NOTE ABOUT GPU USAGE

The workspace provides you with access to a GPU which is necessary for training a transformer model due to the efficiency provided by GPUs on the large amount of computations that are required. To ensure judicious usage of limited resources, please usage the GPU only when you are training the model.

We will keep the maximum length of input to 128 tokens.

In [38]:
```
from torch.utils.data import Dataset

class IMDBDataset(Dataset):
    """
    A custom PyTorch Dataset for the IMDB dataset.
    """

    def __init__(self, data, tokenizer, max_length=MAX_LENGTH):
        self.data = data.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.max_length = max_length
```

```
    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        review = str(self.data.iloc[idx]["review"])
        label = int(self.data.iloc[idx]["label"])

        encoded = self.tokenizer(
            review,
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt"
        )

        input_ids = encoded["input_ids"].squeeze(0)

        return input_ids, label
```

2. Initialize the Dataset

Once the `IMDBDataset` class is defined, we can initialize it directly with the training and validation DataFrames.

In [39]:
```
# Initialize the datasets
train_dataset = IMDBDataset(train_data, tokenizer)
val_dataset = IMDBDataset(val_data, tokenizer)
test_dataset = IMDBDataset(test_df, tokenizer)
```

3. Create a DataLoader

The `DataLoader` class in PyTorch helps manage batches of data during training. We will use it to create training and validation data loaders.

In [40]:
```
from torch.utils.data import DataLoader

# Define batch size
BATCH_SIZE = 32

# Create DataLoader instances
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

You may have seen the use of `RandomShuffler` along with `DataLoader`. That approach is used when you want to train a model over a dataset randomly for a specified number of steps. In this model, we want to use the epoch approach.

An **epoch** is one complete pass through the entire dataset. When using the `DataLoader` in this setup, it ensures that every data point in the dataset is used exactly once during a single epoch. This approach is helpful for training models in a structured manner, ensuring that the

model sees all the training examples and learns from them in each epoch before moving to the next one.

By setting `shuffle=True` for the `train_loader`, the data points are randomly shuffled at the start of each epoch, improving the generalization of the model. You will see this in action later in the code for training the model.

Given below are some assert statements to check your custom dataset and data loader definitions.

In [44]:
```
# Check dataset lengths
assert len(train_dataset) == 22500, "Train dataset length mismatch!"
assert len(val_dataset) == 2500, "Validation dataset length mismatch!"
assert len(test_dataset) == 25000, "Test dataset length mismatch!"

import numpy as np

# Check the first item in the train dataset
input_ids, label = train_dataset[0]

assert isinstance(input_ids, torch.Tensor), "Input IDs should be a torch.Tensor!"
assert isinstance(label, (int, np.integer)), "Label should be an integer or int-lik

# Ensure the input IDs tensor has the correct shape
assert input_ids.shape[0] == train_dataset.max_length, "Input IDs tensor has incorr

print("All dataset and DataLoader checks passed!")
```

```
All dataset and DataLoader checks passed!
```

Customize the Transformer Architecture

In this section, you will customize the transformer architecture to suit the task of binary classification. You may have used a similar architecture in the past for generation tasks. But you will need to make a few tweaks specifically in the `DemoGPT` class to adapt it for the binary classification.

1. Config Dictionary

Your config dictionary bundles all hyperparameters and model settings in one place. Below is the config that we will use in our model:

In [45]:
```
config = {
    "vocabulary_size": tokenizer.vocab_size,  # e.g., ~30522 for bert-base-uncased
    "num_classes": 2,                         # binary classification (pos/neg)
    "d_embed": 128,
    "context_size": MAX_LENGTH,
    "layers_num": 4,
    "heads_num": 4,
```

```
    "head_size": 32,  # 4 heads * 32 = 128 -> matches d_embed
    "dropout_rate": 0.1,
    "use_bias": True
}
```

Key Config Parameters:

- `vocabulary_size` : The total number of tokens in your vocabulary.
- `num_classes` : The number of classes for the classification head (2 = binary).
- `d_embed` : Dimensionality of embeddings (and hidden layers).
- `context_size` : Maximum sequence length for each input.
- `layers_num` : Number of stacked transformer blocks.
- `heads_num` : Number of attention heads in multi-head attention.
- `head_size` : Dimension of each attention head (must satisfy heads_num * head_size = d_embed).
- `dropout_rate` : Probability of dropping units during training to reduce overfitting.
- `use_bias` : Whether linear layers should have bias terms.

2. Class Definitions

Below are the class definitions you will work with. These classes form the core components of the transformer model. You may have seen these before, with the exception of the `DemoGPT` class which will need to be customized.

AttentionHead

In [46]:
```
import torch.nn as nn
import math

class AttentionHead(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.Q_weights = nn.Linear(config["d_embed"], config["head_size"], bias=con
        self.K_weights = nn.Linear(config["d_embed"], config["head_size"], bias=con
        self.V_weights = nn.Linear(config["d_embed"], config["head_size"], bias=con

        self.dropout = nn.Dropout(config["dropout_rate"])

        casual_attention_mask = torch.tril(torch.ones(config["context_size"], confi
        self.register_buffer('casual_attention_mask', casual_attention_mask)

    def forward(self, input):
        batch_size, tokens_num, d_embed = input.shape
        Q = self.Q_weights(input)  # (B, T, head_size)
        K = self.K_weights(input)  # (B, T, head_size)
        V = self.V_weights(input)  # (B, T, head_size)

        # Q @ K^T => (B, T, T)
        attention_scores = Q @ K.transpose(1, 2)
```

```
        # Casual Mask
        attention_scores = attention_scores.masked_fill(
            self.casual_attention_mask[:tokens_num, :tokens_num] == 0,
            float('-inf')
        )
        attention_scores = attention_scores / math.sqrt(K.shape[-1])
        attention_scores = torch.softmax(attention_scores, dim=-1)
        attention_scores = self.dropout(attention_scores)

        return attention_scores @ V
```

Here we use a dummy input aligned with our config:

- Batch size = `BATCH_SIZE` (32)
- Sequence length = `config["context_size"]` (128)
- Embedding dimension = `config["d_embed"]` (128)

In [47]:
```
# Instantiate the AttentionHead
attention_head = AttentionHead(config).to(device)

# Create a dummy input of shape (32, 128, 128)
dummy_input = torch.randn(BATCH_SIZE, config["context_size"], config["d_embed"]).to

# Forward pass
attention_output = attention_head(dummy_input)
print("AttentionHead output shape:", attention_output.shape)
```

```
AttentionHead output shape: torch.Size([32, 128, 32])
```

Expected shape:

> `(B,T,head_size)=(32,128,32)`

MultiHeadAttention

In [48]:
```
class MultiHeadAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        heads_list = [AttentionHead(config) for _ in range(config["heads_num"])]
        self.heads = nn.ModuleList(heads_list)

        self.linear = nn.Linear(config["heads_num"] * config["head_size"], config["
        self.dropout = nn.Dropout(config["dropout_rate"])

    def forward(self, input):
        heads_outputs = [head(input) for head in self.heads]
        x = torch.cat(heads_outputs, dim=-1)  # (B, T, heads_num * head_size)
        x = self.linear(x)                    # (B, T, d_embed)
        x = self.dropout(x)
        return x
```

In [49]:
```
# Instantiate MultiHeadAttention
multi_head_attention = MultiHeadAttention(config).to(device)
```

```
# Same dummy input: (32, 128, 128)
dummy_input = torch.randn(BATCH_SIZE, config["context_size"], config["d_embed"]).to

# Forward pass
mha_output = multi_head_attention(dummy_input)
print("MultiHeadAttention output shape:", mha_output.shape)
```

```
MultiHeadAttention output shape: torch.Size([32, 128, 128])
```

Expected shape:

> (B,T,d_embed)=(32,128,128)

FeedForward

In [50]:
```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.linear_layers = nn.Sequential(
            nn.Linear(config["d_embed"], 4 * config["d_embed"]),
            nn.GELU(),
            nn.Linear(4 * config["d_embed"], config["d_embed"]),
            nn.Dropout(config["dropout_rate"])
        )

    def forward(self, input):
        return self.linear_layers(input)
```

In [51]:
```
# Instantiate FeedForward
feed_forward = FeedForward(config).to(device)

# Dummy input: (32, 128, 128)
dummy_input = torch.randn(BATCH_SIZE, config["context_size"], config["d_embed"]).to

# Forward pass
ff_output = feed_forward(dummy_input)
print("FeedForward output shape:", ff_output.shape)
```

```
FeedForward output shape: torch.Size([32, 128, 128])
```

Expected shape:

> (B,T,d_embed)=(32,128,128)

Block

In [52]:
```
class Block(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.multi_head = MultiHeadAttention(config)
        self.layer_norm_1 = nn.LayerNorm(config["d_embed"])

        self.feed_forward = FeedForward(config)
        self.layer_norm_2 = nn.LayerNorm(config["d_embed"])
```

```
    def forward(self, input):
        x = input
        x = x + self.multi_head(self.layer_norm_1(x))
        x = x + self.feed_forward(self.layer_norm_2(x))
        return x
```

In [53]:
```
# Instantiate a single Block
block = Block(config).to(device)

# Dummy input: (32, 128, 128)
dummy_input = torch.randn(BATCH_SIZE, config["context_size"], config["d_embed"]).to

# Forward pass
block_output = block(dummy_input)
print("Block output shape:", block_output.shape)
```

```
Block output shape: torch.Size([32, 128, 128])
```

Expected shape:

> `(B,T,d_embed)=(32,128,128)`

DemoGPT

Below is the starter code for the `DemoGPT` class, which implements the core of a transformer model tailored for a binary classification task. This implementation builds on the foundation of a transformer architecture and includes the necessary modifications to adapt it for classification.

Key Changes for Binary Classification

To adapt the transformer for classification, the following changes are required:

1. **Add a Classification-Specific Output Layer**:
   - The model needs a linear layer to map the final pooled embeddings to the number of classes.
   - The classification head is implemented using `torch.nn.Linear` with:
     - `in_features` set to `d_embed` (the embedding dimension).
     - `out_features` set to `num_classes` (the number of classes, 2 for binary classification).
     - `bias` set to `False` (optional; bias can be excluded to slightly simplify computations).
2. **Implement a Pooling Mechanism**:
   - Transformers output embeddings for each token in the input sequence. For classification, these token-level embeddings need to be condensed into a single vector.

- A **mean pooling operation** is applied using `torch.mean` across the time dimension (`dim=1`) to aggregate token-level embeddings into a single representation vector.

In [54]:
```python
class DemoGPT(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.token_embedding_layer = nn.Embedding(
            config["vocabulary_size"], config["d_embed"]
        )
        self.positional_embedding_layer = nn.Embedding(
            config["context_size"], config["d_embed"]
        )

        blocks = [
            Block(config) for _ in range(config["layers_num"])
        ]
        self.layers = nn.Sequential(*blocks)
        self.layer_norm = nn.LayerNorm(config["d_embed"])

        # Classification output layer
        self.output_layer = nn.Linear(
            config["d_embed"], config["num_classes"], bias=False
        )

    def forward(self, token_ids):
        batch_size, tokens_num = token_ids.shape

        # Step 1: Token and positional embeddings
        x = self.token_embedding_layer(token_ids)
        positions = torch.arange(tokens_num, device=token_ids.device)
        pos_embed = self.positional_embedding_layer(positions)
        x = x + pos_embed.unsqueeze(0)

        # Step 2: Transformer layers and normalization
        x = self.layers(x)
        x = self.layer_norm(x)

        # Step 3: Mean pooling across the time dimension
        x = x.mean(dim=1)

        # Step 4: Generate classification logits
        logits = self.output_layer(x)

        return logits
```

In [55]:
```python
# Instantiate the model
demo_gpt = DemoGPT(config).to(device)

# Suppose we have a batch of size 32, each with a sequence length of 128
dummy_token_ids = torch.randint(
    0, config["vocabulary_size"],
    (BATCH_SIZE, config["context_size"])
```

```
).to(device)

# Forward pass
logits = demo_gpt(dummy_token_ids)

print("DemoGPT output shape:", logits.shape)
print("Logits sample:\n", logits[:2])  # Print first two examples' logits
```

```
DemoGPT output shape: torch.Size([32, 2])
Logits sample:
 tensor([[ 0.0976, -0.0815],
        [ 0.0029, -0.0438]], device='cuda:0', grad_fn=<SliceBackward0>)
```

Expected shape:

> `(B,num_classes)=(32,2)`

In [57]:
```
# Assert that the number of logits matches the number of classes
assert logits.size(1) == config["num_classes"], (
    f"Expected number of classes {config['num_classes']}, "
    f"but got {logits.size(1)}"
)

# Assert that the batch size of the output matches the input batch size
assert logits.size(0) == BATCH_SIZE, (
    f"Expected batch size {BATCH_SIZE}, "
    f"but got {logits.size(0)}"
)
```

Implement Accuracy Calculation Method

In this section, you will learn how to calculate the validation accuracy for the transformer model on the IMDB dataset. Validation accuracy provides a performance metric that helps assess how well the model generalizes to unseen data during training.

1. Overview

The function to calculate validation accuracy will:

- Evaluate the model on the validation dataset.
- Generate predictions for each batch.
- Compare predictions with the true labels.
- Compute the percentage of correctly classified examples.

2. Key Points

- **Evaluation Mode**: Calling `model.eval()` ensures that dropout and other training-specific layers are disabled during evaluation.

- **No Gradients**: The `torch.no_grad()` context disables gradient computation, reducing memory usage and speeding up validation.
- **Predictions**: `torch.argmax(logits, dim=1)` retrieves the index of the highest logit for each sample, which corresponds to the predicted class label.
- **Accuracy Calculation**: The function computes the fraction of correct predictions out of the total number of samples, then multiplies by 100 to express it as a percentage.

After calculating the validation accuracy, incorporate this function into your training loop. Typically, you would call calculate_accuracy at the end of each epoch or after a specific number of training steps. Monitoring validation accuracy over time helps you track performance gains and identify potential overfitting or underfitting issues.

In [58]:
```
def calculate_accuracy(model, data_loader, device):
    """
    Calculate the accuracy of the model on the validation dataset.
    """
    model.eval()
    total_correct = 0
    total_samples = 0

    with torch.no_grad():
        for input_ids, labels in data_loader:
            input_ids = input_ids.to(device)
            labels = labels.to(device)

            logits = model(input_ids)
            predictions = torch.argmax(logits, dim=1)

            total_correct += (predictions == labels).sum().item()
            total_samples += labels.size(0)

    accuracy = (total_correct / total_samples) * 100
    return accuracy
```

In [59]:
```
model = DemoGPT(config).to(device)
```

In [60]:
```
validation_accuracy = calculate_accuracy(model, val_loader, device)
print(f"Validation Accuracy: {validation_accuracy:.2f}%")
```

```
Validation Accuracy: 50.04%
```

As you can see, the validation accuracy is close to 50%. This is expected as the model's parameters have been initialized randomly.

Train the Model

In this section, we will define the training loop for the transformer-based model designed for sentiment analysis. The training loop is crucial for optimizing the model's weights and biases to minimize the loss function and improve classification performance.

Training Loop

The training loop will involve the following steps:

1. **Iterate through epochs**: Repeat the training process for a predefined number of epochs.
2. **Load batches of data**: Use the `DataLoader` to retrieve batches of input IDs and labels.
3. **Forward pass**: Compute the logits by passing the input IDs through the model.
4. **Compute loss**: Use cross-entropy loss as the criterion.
5. **Backward pass and optimization**: Backpropagate the loss and update the model parameters using the optimizer.
6. **Validation**: Calculate the validation accuracy after each epoch.

In [64]:
```
import torch
import torch.optim as optim

# Define the device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# Training parameters
EPOCHS = 5

# Initialize model, loss, and optimizer
model = DemoGPT(config).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=3e-4)

# Track the best validation accuracy
best_val_accuracy = 0.0
checkpoint_path = "best_sentimentscope_model.pth"

# Training loop
for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    for step, (input_ids, labels) in enumerate(train_loader):
        input_ids = input_ids.to(device)
        labels = labels.to(device)

        # Forward pass
        logits = model(input_ids)

        # Calculate loss
        loss = criterion(logits, labels)

        # Backpropagation and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
```

```
        # Print progress every 100 batches
        if (step + 1) % 100 == 0:
            print(
                f"Epoch [{epoch + 1}/{EPOCHS}], "
                f"Step [{step + 1}/{len(train_loader)}], "
                f"Loss: {running_loss / 100:.4f}"
            )
            running_loss = 0.0

    # Evaluate on validation data
    val_accuracy = calculate_accuracy(model, val_loader, device)
    print(
        f"Epoch {epoch + 1}/{EPOCHS} - "
        f"Validation Accuracy: {val_accuracy:.2f}%"
    )

    # Save the best model based on validation accuracy
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(
            {
                "model_state_dict": model.state_dict(),
                "config": config,
                "best_val_accuracy": best_val_accuracy,
                "epoch": epoch + 1,
            },
            checkpoint_path,
        )
        print(f"Best model saved: {checkpoint_path}")

print(f"\nBest validation accuracy: {best_val_accuracy:.2f}%")
print(f"Checkpoint: {checkpoint_path}")
```

```
Using device: cuda
Epoch [1/5], Step [100/704], Loss: 0.6877
Epoch [1/5], Step [200/704], Loss: 0.6772
Epoch [1/5], Step [300/704], Loss: 0.6519
Epoch [1/5], Step [400/704], Loss: 0.6127
Epoch [1/5], Step [500/704], Loss: 0.5816
Epoch [1/5], Step [600/704], Loss: 0.5787
Epoch [1/5], Step [700/704], Loss: 0.5548
Epoch 1/5 - Validation Accuracy: 71.52%
Best model saved: best_sentimentscope_model.pth
Epoch [2/5], Step [100/704], Loss: 0.5233
Epoch [2/5], Step [200/704], Loss: 0.5054
Epoch [2/5], Step [300/704], Loss: 0.4963
Epoch [2/5], Step [400/704], Loss: 0.4890
Epoch [2/5], Step [500/704], Loss: 0.4912
Epoch [2/5], Step [600/704], Loss: 0.4931
Epoch [2/5], Step [700/704], Loss: 0.4800
Epoch 2/5 - Validation Accuracy: 76.28%
Best model saved: best_sentimentscope_model.pth
Epoch [3/5], Step [100/704], Loss: 0.4374
Epoch [3/5], Step [200/704], Loss: 0.4340
Epoch [3/5], Step [300/704], Loss: 0.4218
Epoch [3/5], Step [400/704], Loss: 0.4275
Epoch [3/5], Step [500/704], Loss: 0.4074
Epoch [3/5], Step [600/704], Loss: 0.4154
Epoch [3/5], Step [700/704], Loss: 0.4157
Epoch 3/5 - Validation Accuracy: 78.40%
Best model saved: best_sentimentscope_model.pth
Epoch [4/5], Step [100/704], Loss: 0.3607
Epoch [4/5], Step [200/704], Loss: 0.3773
Epoch [4/5], Step [300/704], Loss: 0.3748
Epoch [4/5], Step [400/704], Loss: 0.3636
Epoch [4/5], Step [500/704], Loss: 0.3678
Epoch [4/5], Step [600/704], Loss: 0.3757
Epoch [4/5], Step [700/704], Loss: 0.3605
Epoch 4/5 - Validation Accuracy: 78.16%
Epoch [5/5], Step [100/704], Loss: 0.3399
Epoch [5/5], Step [200/704], Loss: 0.3130
Epoch [5/5], Step [300/704], Loss: 0.3148
Epoch [5/5], Step [400/704], Loss: 0.3222
Epoch [5/5], Step [500/704], Loss: 0.3263
Epoch [5/5], Step [600/704], Loss: 0.3069
Epoch [5/5], Step [700/704], Loss: 0.3360
Epoch 5/5 - Validation Accuracy: 80.00%
Best model saved: best_sentimentscope_model.pth

Best validation accuracy: 80.00%
Checkpoint: best_sentimentscope_model.pth
```

Notes

- The `evaluate_accuracy` function calculates the model's accuracy on the validation dataset. Ensure this function is defined and works as expected.
- The training progress is logged every 100 steps to monitor performance.

- After each epoch, the validation accuracy is printed to ensure the model generalizes well to unseen data.

Test the Model

In this section, you will evaluate the performance of your trained transformer model on the test dataset. Testing the model involves loading the test dataset, passing it through the model, and calculating the accuracy.

In [65]:
```
test_accuracy = calculate_accuracy(model, test_loader, device)
print(f"Test Accuracy: {test_accuracy:.2f}%")
```

```
Test Accuracy: 77.50%
```

In [2]:
```
import os
import zipfile

files_to_include = [
    "SentimentScope_starter.ipynb",
    "best_sentimentscope_model.pth"
]

zip_name = "SentimentScope_Project.zip"

with zipfile.ZipFile(zip_name, "w", zipfile.ZIP_DEFLATED) as zipf:
    for file in files_to_include:
        if os.path.exists(file):
            zipf.write(file)
            print("Added:", file)
        else:
            print("Missing:", file)

print("ZIP created:", os.path.abspath(zip_name))
```

```
Added: SentimentScope_starter.ipynb
Added: best_sentimentscope_model.pth
ZIP created: /workspace/starter/SentimentScope_Project.zip
```

In [3]:
```
from IPython.display import FileLink, display

display(FileLink("SentimentScope_Project.zip"))
```

SentimentScope_Project.zip

With the accuracy calculated, you can verify if the model meets the project goal of achieving greater than 75% accuracy on the test dataset.

Try training your model for more than 3 epochs! Try increasing the size of the embedding used in the model. Or try increasing the number of blocks or layers in the model. You may be able to improve the accuracy of your model further!

Conclusion

Summary

SentimentScope is a sentiment analysis project that uses a custom Transformer-based neural network to classify IMDB movie reviews as positive or negative. The IMDB dataset was used for training, validation, and testing, with text reviews tokenized using the BERT tokenizer (`bert-base-uncased`) and padded or truncated to a maximum sequence length of 128 tokens.

The model was built using token and positional embeddings, multi-head self-attention, feed-forward neural networks, and a final classification layer. It was trained from scratch using PyTorch, the AdamW optimizer, and cross-entropy loss for five epochs on a CUDA-enabled GPU.

During training, the model achieved a **best validation accuracy of 80.00% in Epoch 5**, exceeding the project's target of 75%. The best-performing model checkpoint was saved as `best_sentimentscope_model.pth` for reuse and inference.

Key Takeaways

1. **Transformer-based sentiment classification:** The project demonstrates how a custom Transformer model can learn to classify movie reviews as positive or negative using token embeddings, positional embeddings, and multi-head self-attention.
2. **Training and model evaluation:** Training for five epochs improved validation performance to 80.00%, surpassing the target accuracy of 75%. Saving the best checkpoint enables the trained model to be loaded for future predictions without retraining.